IDOR (Insecure Direct Object Reference) Report

1. Room Name

TryHackMe: IDOR

2. Objective

To understand and exploit Insecure Direct Object Reference (IDOR) vulnerabilities where an application exposes internal object references without proper access control.

3. Vulnerability / Concept

IDOR occurs when an application exposes a direct reference to an internal object and fails to verify whether the requesting user is authorized to access it.

IDOR occurs when:

- An application uses user-controllable input to access objects directly.
- No proper authorization check is performed.
- An attacker can access resources belonging to other users.

Examples:

- Changing `user_id=123` to `user_id=124`.
- Changing `invoice=1001` to `invoice=1002`.
- Accessing `/admin/user/5` directly.

The vulnerability exists because:

- The application trusts user input.
- No ownership verification is performed.
- Direct object references are exposed.

---

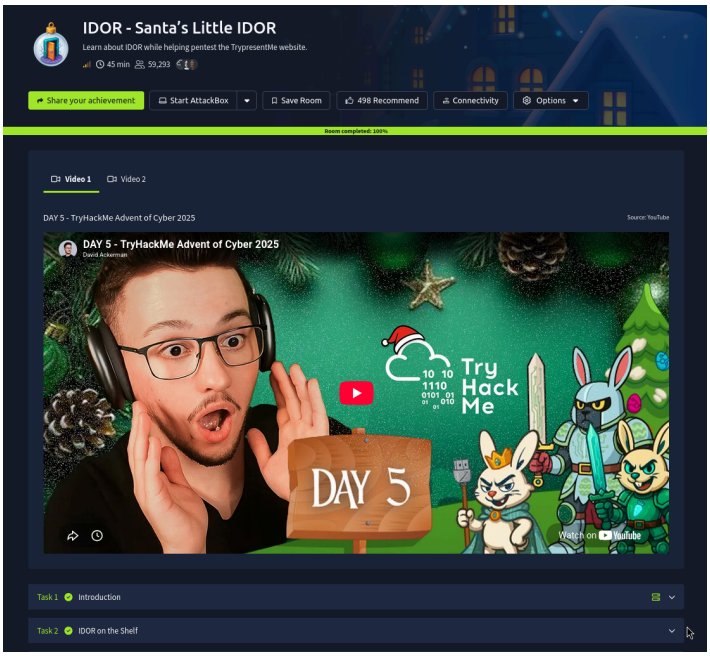
4. Testing Approach

1. Identify parameters that reference objects, such as IDs and filenames.
2. Modify the ID value to another user's ID.
3. Observe whether unauthorized data is returned.
4. Confirm the IDOR vulnerability.
5. Document the impact.

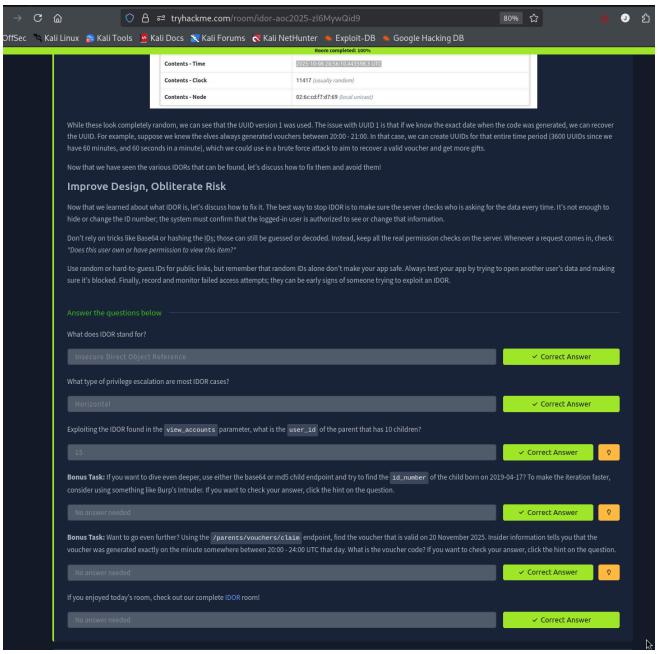
---

5. Evidence / Screenshots

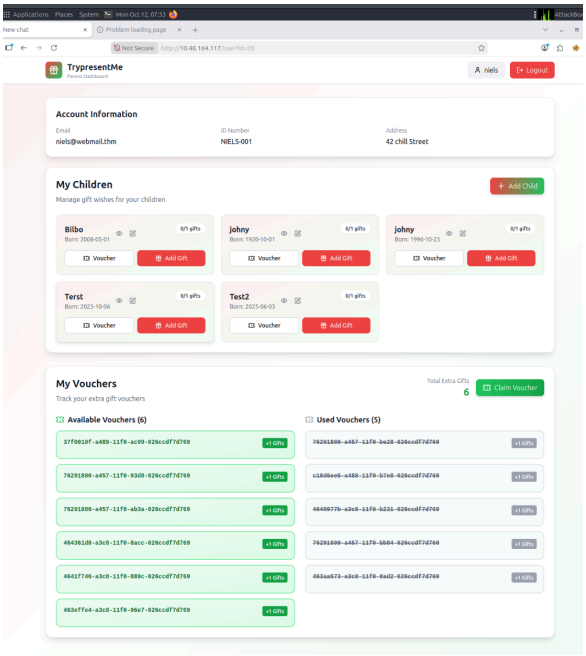
1. IDOR room completion.



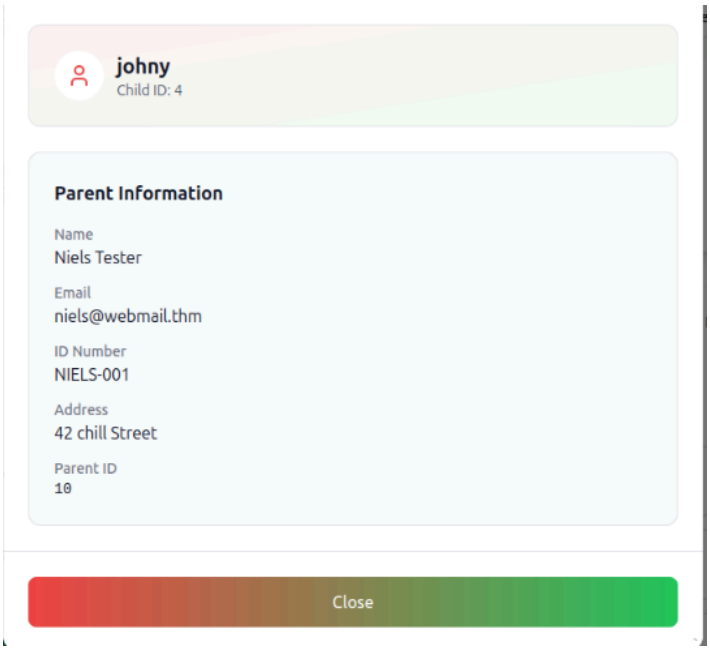
2. Correct answers in the room



3. ID parameter being changed.



4. Unauthorized access to another user's data.



---

6. Impact

An IDOR vulnerability can lead to the following consequences:

- Unauthorized access to sensitive data.
- Ability to view, modify, or delete other users' information.
- Privacy breach.
- Account takeover.
- Data theft.
- Regulatory compliance issues, including GDPR-related concerns.

---

7. Remediation

The following controls should be implemented to reduce the risk of IDOR vulnerabilities:

- Implement proper access-control checks.
- Verify user ownership on every request.
- Use indirect references through mapping tables.
- Use UUIDs instead of sequential IDs where appropriate.
- Apply deny-by-default authorization.
- Log and monitor access attempts.

---

8. What You Learned

- IDOR is a common and dangerous vulnerability.
- Direct object references can be manipulated.
- Access control must be enforced server-side.
- User ownership must always be verified.
- Sequential IDs are easy to guess.
- UUIDs provide better security, although they do not replace authorization checks.

Conclusion

Preventing IDOR requires server-side authorization and ownership validation for every object access. These controls help ensure that users can access only the resources they are permitted to use and support a clear, defensible approach to secure application design.
